week 9 - LoRA

arxiv.org
https://arxiv.org/pdf/2106.09685

1. Abstract

- NLP에서 중요한 패러다임
  : 일반 도메인 데이터에 대한 대규모 사전학습(pre-training)
  + 이후 특정 태스크나 도메인에 대한 적응(adaptation) → 파인튜닝
- 방대한 파라미터를 가진 독립적인 파인튜닝 모델 인스턴스 배포

  → 비용 매우 大

💡**제안**
사전학습된 모델의 가중치를 고정(freeze)
+ 트랜스포머 아키텍처의 각 레이어에 학습 가능한 저계수(rank) 분해 행렬을 주입
→ 다운스트림 태스크에 필요한 학습 가능 파라미터의 수를 크게 줄이는 방법

2. Introduction

- 기존 모델 배포 방식
  - 소수 태스크별 파라미터만 저장, 로드
    → 배포 시 운영 효율성이 크게 향상

하지만,

  - 모델 깊이 늘어남 → 추론 지연 시간 生
  - 모델이 사용 가능한 시퀀스 길이 줄임

→ 파인 튜닝 baseline 성능에 미치지 못해, 효율성과 모델 품질 사이의 트레이드 오프 야기

과매개변수화(over-parametrized)된 학습 모델이 실제로는 낮은 내재적 차원(low intrinsic dimension) 위에 존재하는 연구에서 영감을 얻음.
→ **모델 적응 과정에서 가중치의 변화 역시 낮은 "내재적 랭크(intrinsic rank)"를 가진다고 가정**

- 핵심 장점
  - 사전 학습된 하나의 모델 공유 → 다른 태스크를 위한 작은 LoR모듈 만드는데 사용 O

    👍 태스크 전환 효율적/저장 공간 요구량과 태스크 전화 오버헤드 ↓
  - 적응형 옵티마이저 사용 → 대부분의 파라미터 계산 無, 옵티마이저 상태 유지 필요 X

3. Related work

딥러닝에서의 Low-Rank 구조 관련 연구

- 학습 시점부터 저계수 제약 명시적으로 부과

  → 고정된 모델에 저계수 업데이트 적용해 적응시키는 것과 차이 有
- 어댑터 레이어를 추가하는 방식/입력층 활성화(activation)의 일부 형태를 최적화하는 방식
  - 두 전략 모두, 특히 대규모이면서 지연(latency)에 민감한 프로덕션 환경에서는 한계 有

⭐ **추론 지연 無 + 시퀀스 길이 손실 無 → 이 두가지 문제 동시 해결**

4. Method

1. low-rank로 매개변수화된 업데이트 행렬

- 적응 과정에서 가중치 행렬에 대한 누적 그래디언트 없데이트가 풀랭크일 필요 X
  - LoRA를 모든 가중치 행렬에 적용, 모든 bias 학습시킨다면,
    : 랭크 r을 사전학습된 가중치 행렬로 설정
    → 완전 파인튜닝의 표현력을 대략적으로 회복 O
  - 학습 가능한 파라미터 수 ↑ → LoRA학습은 원래 모델 학습 하는 것으로 수렴

2. Transformer에 적용

- 적용
  - Transformer
    - 셀프 어텐션 모듈 4개
  - MLP
    - 2개 모듈

  (출력 차원이 어텐션 헤드로 나뉘어 있지만, dmodel x dmodel 차원의 단일 행렬로 취급)

  👍 **메모리 및 저장 공간 사용량의 감소**

3. Experiments

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB_{base} (FT)*	125.0M	**87.6**	94.8	90.2	**63.6**	92.8	**91.9**	78.7	91.2	86.4
RoB_{base} (BitFit)*	0.1M	84.7	93.7	**92.7**	62.0	91.8	84.0	81.5	90.8	85.2
RoB_{base} ($Adpt^D$)*	0.3M	$87.1_{\pm.0}$	$94.2_{\pm.1}$	$88.5_{\pm1.1}$	$60.8_{\pm.4}$	$93.1_{\pm.1}$	$90.2_{\pm.0}$	$71.5_{\pm2.7}$	$89.7_{\pm.3}$	84.4
RoB_{base} ($Adpt^D$)*	0.9M	$87.3_{\pm.1}$	$94.7_{\pm.3}$	$88.4_{\pm.1}$	$62.6_{\pm.9}$	$93.0_{\pm.2}$	$90.6_{\pm.0}$	$75.9_{\pm2.2}$	$90.3_{\pm.1}$	85.4
RoB_{base} (LoRA)	0.3M	$87.5_{\pm.3}$	$\mathbf{95.1}_{\pm.2}$	$89.7_{\pm.7}$	$63.4_{\pm1.2}$	$\mathbf{93.3}_{\pm.3}$	$90.8_{\pm.1}$	$\mathbf{86.6}_{\pm.7}$	$\mathbf{91.5}_{\pm.2}$	**87.2**
RoB_{large} (FT)*	355.0M	90.2	**96.4**	**90.9**	68.0	94.7	**92.2**	86.6	92.4	88.9
RoB_{large} (LoRA)	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.9}_{\pm1.2}$	$\mathbf{68.2}_{\pm1.9}$	$\mathbf{94.9}_{\pm.3}$	$91.6_{\pm.1}$	$\mathbf{87.4}_{\pm2.5}$	$\mathbf{92.6}_{\pm.2}$	**89.0**
RoB_{large} ($Adpt^P$)†	3.0M	$90.2_{\pm.3}$	$96.1_{\pm.3}$	$90.2_{\pm.7}$	$\mathbf{68.3}_{\pm1.0}$	$\mathbf{94.8}_{\pm.2}$	$\mathbf{91.9}_{\pm.1}$	$83.8_{\pm2.9}$	$92.1_{\pm.7}$	88.4
RoB_{large} ($Adpt^P$)†	0.8M	$\mathbf{90.5}_{\pm.3}$	$\mathbf{96.6}_{\pm.2}$	$89.7_{\pm1.2}$	$67.8_{\pm2.5}$	$\mathbf{94.8}_{\pm.3}$	$91.7_{\pm.2}$	$80.1_{\pm2.9}$	$91.9_{\pm.4}$	87.9
RoB_{large} ($Adpt^H$)†	6.0M	$89.9_{\pm.5}$	$96.2_{\pm.3}$	$88.7_{\pm2.9}$	$66.5_{\pm4.4}$	$94.7_{\pm.2}$	$92.1_{\pm.1}$	$83.4_{\pm1.1}$	$91.0_{\pm1.7}$	87.8
RoB_{large} ($Adpt^H$)†	0.8M	$90.3_{\pm.3}$	$96.3_{\pm.5}$	$87.7_{\pm1.7}$	$66.3_{\pm2.0}$	$94.7_{\pm.2}$	$91.5_{\pm.1}$	$72.9_{\pm2.9}$	$91.5_{\pm.5}$	86.4
RoB_{large} (LoRA)†	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.2}_{\pm1.0}$	$68.2_{\pm1.9}$	$\mathbf{94.8}_{\pm.3}$	$91.6_{\pm.2}$	$\mathbf{85.2}_{\pm1.1}$	$\mathbf{92.3}_{\pm.5}$	**88.6**
DeB_{XXL} (FT)*	1500.0M	91.8	**97.2**	92.0	72.0	**96.0**	92.7	93.9	92.9	91.1
DeB_{XXL} (LoRA)	4.7M	$\mathbf{91.9}_{\pm.2}$	$96.9_{\pm.2}$	$\mathbf{92.6}_{\pm.6}$	$\mathbf{72.4}_{\pm1.1}$	$\mathbf{96.0}_{\pm.1}$	$\mathbf{92.9}_{\pm.1}$	$\mathbf{94.9}_{\pm.4}$	$\mathbf{93.0}_{\pm.2}$	**91.3**

Table 2: RoBERTabase, RoBERTalarge, and DeBERTaXXL with different adaptation methods on the GLUE benchmark.

- LoRA를 적용하였을 때, Avg 향상을 보임

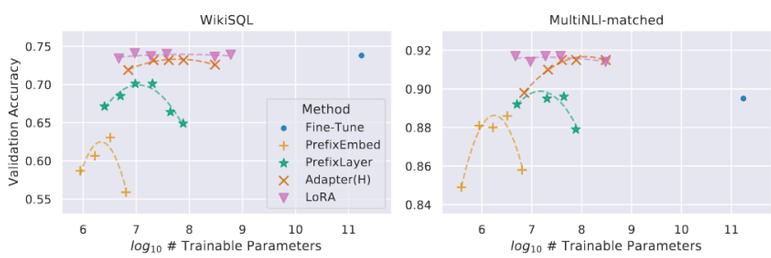


Figure 2: GPT-3 175B validation accuracy vs. number of trainable parameters of several adaptation methods on WikiSQL and MNLI-matched. LoRA exhibits better scalability and task performance.

- LoRA는 세 가지 데이터셋 모두에서 미세조정 베이스라인과 동등하거나 이를 능가
- 모든 방법이 학습 가능한 파라미터를 늘린다고 해서 단조적으로 성능이 좋아지는 것은 아님
- Prefix, PrefixLayer 임베딩 튜닝에서 일정 수 이상의 특수 토큰을 사용할 경우,
  성능이 크게 떨어지는 것을 확인

  → 특수 토큰이 많아질수록 입력 분포가 사전학습 데이터 분포로부터 더 멀리 벗어나게 되기 때문일 것으로 추정 (논문)

4. Conclusion

1. 한계점

- fine-tuning이나 LoRA의 기저 메커니즘은 아직 명확하지 않음

- LoRA를 적용할 가중치 행렬 선택 시, 휴리스틱에 의존
    - 더 원칙에 기반한 방법 모색 필요

2. 결론

- 거대 언어 모델 fine-tuning
  → 필요한 하드웨어, 저장/전환 비용 측면에서 매우 비쌈
- **높은 모델 품질 유지 + 추론 지연 生 X + 입력 시퀀스 축소 無**
  → 효율적인 적응 전략을 제안함
- dense layer를 가진 모든 신경망에 일반적으로 적용 O